



دانشگاه صنعتی شریف

دانشکده مهندسی مکانیک

طراحی مسیر و جلوگیری از برخورد به موانع

تمرین ششم درس رباتیک

هانیه زهرا بوداگی - ۹۹۱۰۵۹۵۶

استاد

دکتر بهزادی پور

ترم اول سال ۱۴۰۲

فهرست مطالب

۱. صورت سوالات	1
۱.۱ سوال اول	1
۲.۱ سوال دوم	1
۳.۱ سوال سوم	1
۴.۱ سوال چهارم (امتیازی)	2
۲. پاسخ سوالات	2
۱.۲ سوال اول	2
۲.۲ سوال دوم	4
۳.۲ سوال سوم	5
۴.۲ سوال چهارم	9

۱. صورت سوالات

در این مسئله از ما خواسته می‌شود تا با استفاده از روش میدان پتانسیل، مسیر حرکت عملگر نهایی یک ربات را در یک صفحه‌ی دو بعدی با موانع از پیش تعیین شده طراحی نماییم.

۱.۱ سوال اول

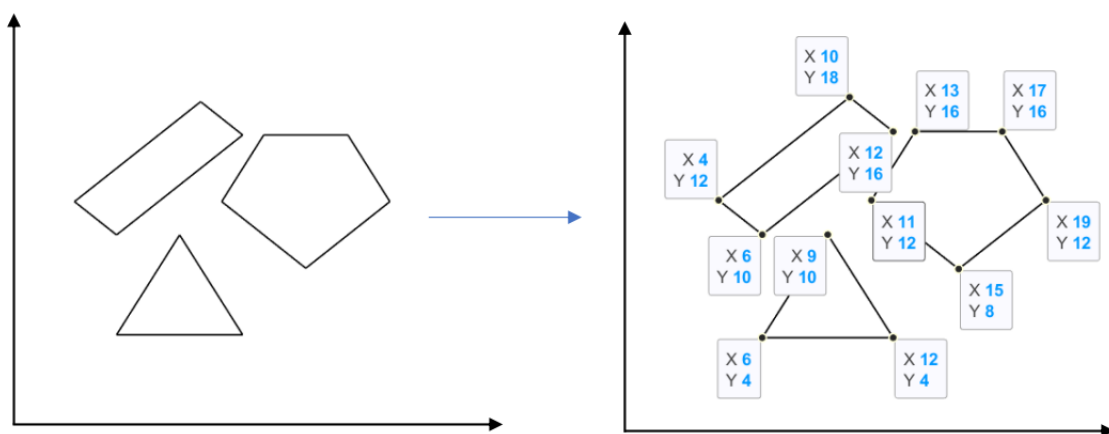
در بخش اول مسئله، باید تابعی در متلب نوشته شود که نقاط مسیر را تولید می‌کند. ورودی‌های این تابع، نقطه‌ی شروع و پایان حرکت، η به عنوان ضریب scale میدان جاذبه و ماتریس B هستند. در ماتریس B، داده‌های مربوط به موانع وجود دارد. تا زمانی که آخرین نقطه‌ی تولید شده توسط تابع با نقطه‌ی پایان به فاصله‌ی ۰/۱ نرسیده باشد، تابع به تولید نقاط مسیر ادامه خواهد داد. خروجی این تابع، ماتریس P می‌باشد که دارای داده‌های مختصات نقاط مسیر است.

`function P = Path_generator (Xs, Xf, eta, B)`

۲.۱ سوال دوم

در این قسمت، مثالی داده می‌شود تا تابع خود را توسط آن محک بزنیم. $\eta = \alpha = 1$ در نظر گرفته شده و مقدار متغیر اپسیلون نیز برابر با ۰/۱ می‌باشد (که گام‌های مسیر را تعیین می‌کند). مختصات رئوس موانع در شکل زیر داده شده است. مختصات نقاط شروع و پایان نیز به شرح زیر می‌باشد:

$$X_s = (1,10), X_f = (22,12)$$



۳.۱ سوال سوم

مانع جدیدی به شکل یک خط بین دو نقطه‌ی (۱۷/۵، ۸) و (۱۷/۵، ۱۰) اضافه می‌کنیم. حال باید تابع را مجدداً امتحان کنیم. سوال پرسیده شده که مسیر در کجا گیر می‌کند.

۴.۱ سوال چهارم (امتیازی)

در این بخش خواسته شده که از یک الگوریتم Random Walk پیاده‌سازی کرده و مشکل بخش سوم مسئله را با استفاده از آن حل بنماییم. این الگوریتم زمانی وارد عمل خواهد شد که الگوریتم میدان پتانسیل (PFA) در یک نقطه‌ی مینیمم محلی گیر کرده باشد. سپس، با استفاده از Random Walk، یک پرش با ماکزیمم اندازه‌ی a خواهیم داشت که در این مسئله، a را برابر با ۶ قرار می‌دهیم. زاویه‌ی پرش رندم می‌باشد مادامی که مطمئن باشیم با هیچ مانعی برخورد نخواهیم داشت. می‌توان روش دیگری برای تعیین زاویه‌ی بهینه‌ی فرار از نقطه‌ی مینیمم محلی نیز ارائه داد. سپس، نقطه‌ی به دست آمده با این روش به الگوریتم میدان پتانسیل باز می‌گردد و بررسی می‌شود که آیا می‌توان با استفاده از آن مسیر را تا نقطه‌ی پایانی ادامه داد یا خیر.

۲. پاسخ سوالات

۱.۲ سوال اول

ماتریس B را به شکل یک متغیر نوع $Cell_{2 \times n}$ در متلب تعریف می‌کنیم، به این صورت که هر ستون آن، دارای داده‌های یک مانع از مانع ۱ تا مانع n باشد. مولفه‌ی قرار گیرنده در $B\{1, i\}$ نمایانگر تعداد اضلاع مانع بوده و مولفه‌ی $B\{2, i\}$ ، ماتریسی است که مختصات رئوس مانع را به ترتیب اتصال به هم یکی پس از دیگری در خود جای داده است.

الگوریتم میدان پتانسیل را به شکل توضیح داده شده در جزوه‌ی درسی در متلب پیاده‌سازی می‌کنیم. کد نهایی به شرح زیر می‌باشد:

```
function P = Path_generator(Xs, Xf, eta, B)

%Xs is the starting point
%Xf is the finishing point
%eta is our chosen scaling factor
%B is the cell in which we have stored obstacle data

k = 1; %just a variable to keep track of the number of the points in the path
X = []; %initiating the path points matrix
X(:,k) = Xs; %starting point
[~,n] = size(B); %n: the number of the obstacles

while norm(Xf-X(:,k)) > 0.1 %the threshold we chose to get to Xf
    x = X(:,k); %making things a shade simpler
    F_att = -eta*(x-Xf); %attraction force calculated
    F_rep = zeros(2,n);

    for i = 1:n %we're gonna find Bi for n objects
        m = B{1,i}; %m: number of the points of the object i-th
        points = B{2,i}; %coordinates of those points
        B_dis = ones(2,m);
        for j = 1:m
            p1 = points(:,j);
            if j == m
```

```

        p2 = points(:,1);
    else
        p2 = points(:,j+1);
    end

    if m == 1
        p2 = points(:,2);
    end

    side = p2 - p1; %one side of the polygon
    rot_mat = [0 -1; 1 0]; %rotation matrix (for 90 degrees)
    D = [side, rot_mat*side];
    coeff = D\(x - p1); %coeff consists of alpha and beta
    alpha = coeff(1); %we just needed one of them, so we choose alpha

    if alpha > 0 && alpha < 1 %finding where Bi of this side lies on
        Bf = alpha*(p2 - p1) + p1;
    elseif alpha <= 0
        Bf = p1;
    else
        Bf = p2;
    end

    B_dis(:,j) = Bf; %Bi for one side found, got called Bf, and saved
end

Bi = B_dis(:,1); %comparing all Bi's to find the closest one to x
if m > 1
    for j = 2:m
        if norm(x - B_dis(:,j)) < norm(x - Bi)
            Bi = B_dis(:,j);
        end
    end
end
% Bi found

ro_i = norm(Bi - x); %calculating the repulsive forces
alpha_i = 1;
ro_o = 2; %alpha_i and ro_o are the same for all obstacles
if ro_i <= ro_o
    F_rep(:,i) = alpha_i*(1/ro_i^3)*((1/ro_i) - (1/ro_o))*(x-Bi);
end
end

sumFrep = zeros(2,1); %summing up the repulsive forces now
for t = 1:n
    sumFrep(1) = sumFrep(1) + F_rep(1,t);
    sumFrep(2) = sumFrep(2) + F_rep(2,t);
end

F = F_att + sumFrep; %total force be like:
Fd = F / norm(F); %here comes the unit vector of the path we pave
eps = 0.1; %and here is the step
X(:, k+1) = x + eps*Fd; %next point of the path
k = k+1; %let's do it all over

end

```

```

X(:,k) = Xf; %adding our last point to the path
P = X; %path generated

end

```

۲.۲ سوال دوم

سل B را تعریف کرده و مطابق تعریفی که در بخش اول داشتیم، داده‌های مسئله را در آن قرار می‌دهیم:

```

clc; clear;
Obst = [3,4,5]; %this vectors contains the number of sides for each obstacle
n = length(Obst); % n = number of the obstacles

% saving obstacles data in a cell
B = cell(2,n);
B{1,1} = Obst(1); B{2,1} = [9 6 12; 10 4 4];
B{1,2} = Obst(2); B{2,2} = [4 10 12 6; 12 18 16 10];
B{1,3} = Obst(3); B{2,3} = [11 13 17 19 15; 12 16 16 12 8];

```

سپس، نقاط ابتدایی و انتهایی مسیر را تعریف می‌کنیم و متغیر η را مقداردهی می‌نماییم.

پس از آن، با استفاده از تابعی که در بخش اول نوشتیم، مسیر را طراحی می‌کنیم و مختصات نقاط مسیر را به عنوان ماتریس P دریافت می‌کنیم.

برای کشیدن موانع، از تابع `polyshape` استفاده می‌کنیم و هر مانع را با توجه به مختصات آن، به شکل یک شیء چند ضلعی ذخیره کرده و سپس با استفاده از دستور `plot` آن‌ها را رسم می‌کنیم.

در نهایت، مختصات نقاط X و Y مسیر را جداگانه ذخیره کرده و آن‌ها را نیز روی شکلمان رسم می‌کنیم.

کد کامل اسکریپت نهایی برای این بخش سوال و تصویر موانع و مسیر طراحی شده به شرح زیر می‌باشد:

```

clc; clear;
Obst = [3,4,5]; %this vectors contains the number of sides for each obstacle
n = length(Obst); % n = number of the obstacles

% saving obstacles data in a cell
B = cell(2,n);
B{1,1} = Obst(1); B{2,1} = [9 6 12; 10 4 4];
B{1,2} = Obst(2); B{2,2} = [4 10 12 6; 12 18 16 10];
B{1,3} = Obst(3); B{2,3} = [11 13 17 19 15; 12 16 16 12 8];

Xs = [1;10];
Xf = [22;12];
eta = 1;

P = Path_generator(Xs,Xf,eta,B);

% plotting obstacles
obs1 = polyshape(B{2,1}(1,:), B{2,1}(2,:));

```

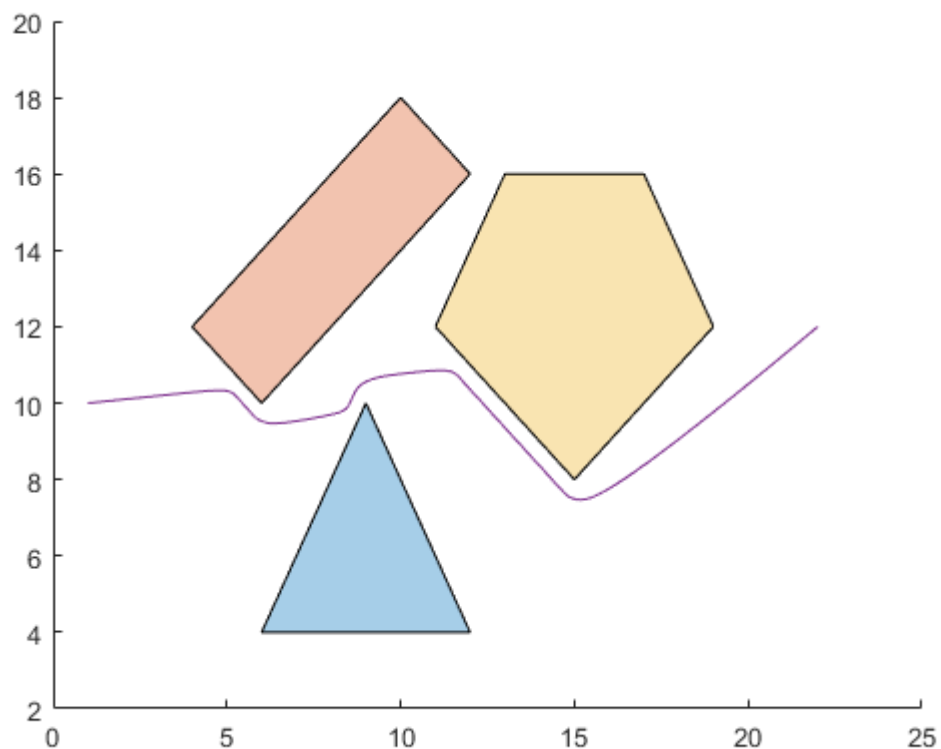
```

obs2 = polyshape(B{2,2}(1,:), B{2,2}(2,:));
obs3 = polyshape(B{2,3}(1,:), B{2,3}(2,:));

plot(obs1);
hold on
plot(obs2);
plot(obs3);

path_x = P(1,:); path_y = P(2,:);
plot(path_x, path_y)

```



۳.۲ سوال سوم

در یک اسکریپت جدید، مجدداً سل B را تعریف کرده و این بار، مختصات نقاط داده شده را نیز در آن قرار می‌دهیم. در واقع، مانعی با یک ضلع تعریف می‌کنیم. سپس دیتا را مجدداً به تابع طراحی مسیر ورودی داده و مشاهده می‌کنیم که تابع وارد لوپ بینهایت شده و هیچگاه از while خارج نمی‌شود:

اسکریپت نوشته شده:

```

Obst2 = [3,4,5,1]; %this vectors contains the number of sides for each obstacle
n2 = length(Obst2); % n = number of the obstacles

% saving obstacles data in a cell
B2 = cell(2,n2);
B2{1,1} = Obst2(1); B2{2,1} = [9 6 12; 10 4 4];
B2{1,2} = Obst2(2); B2{2,2} = [4 10 12 6; 12 18 16 10];

```

```
B2{1,3} = Obst2(3); B2{2,3} = [11 13 17 19 15; 12 16 16 12 8];
B2{1,4} = Obst2(4); B2{2,4} = [17.5 17.5;10 8];
```

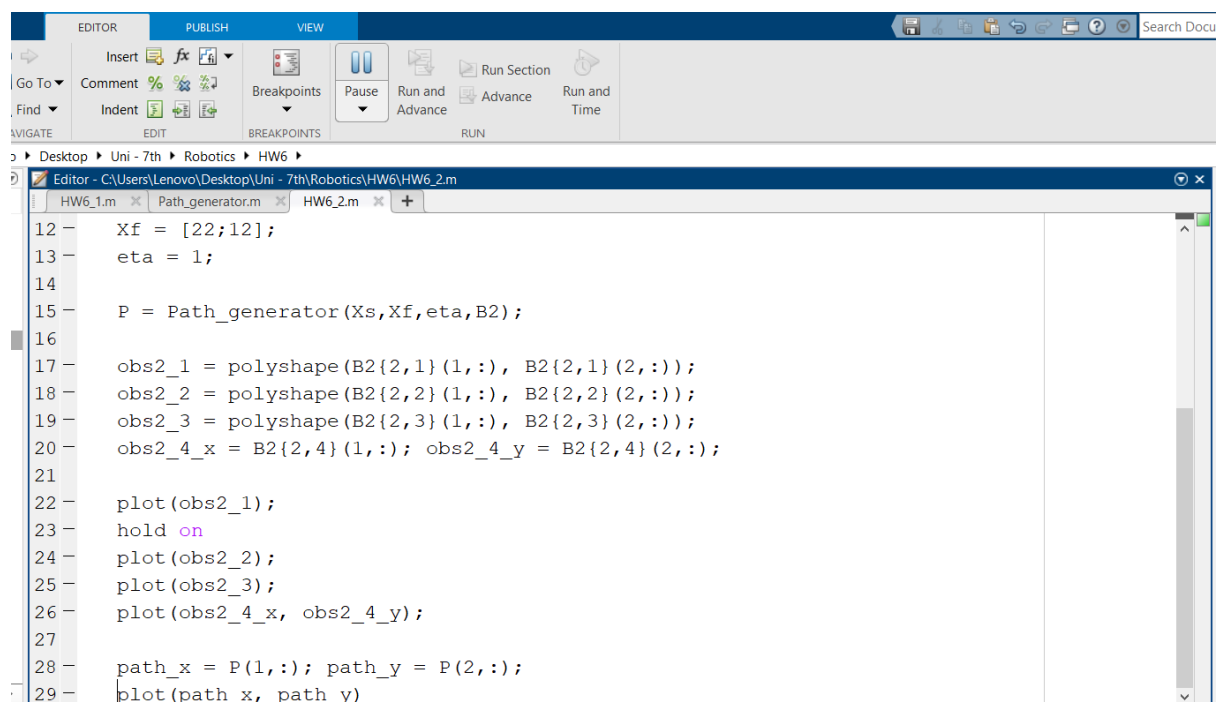
```
Xs = [1;10];
Xf = [22;12];
eta = 1;
```

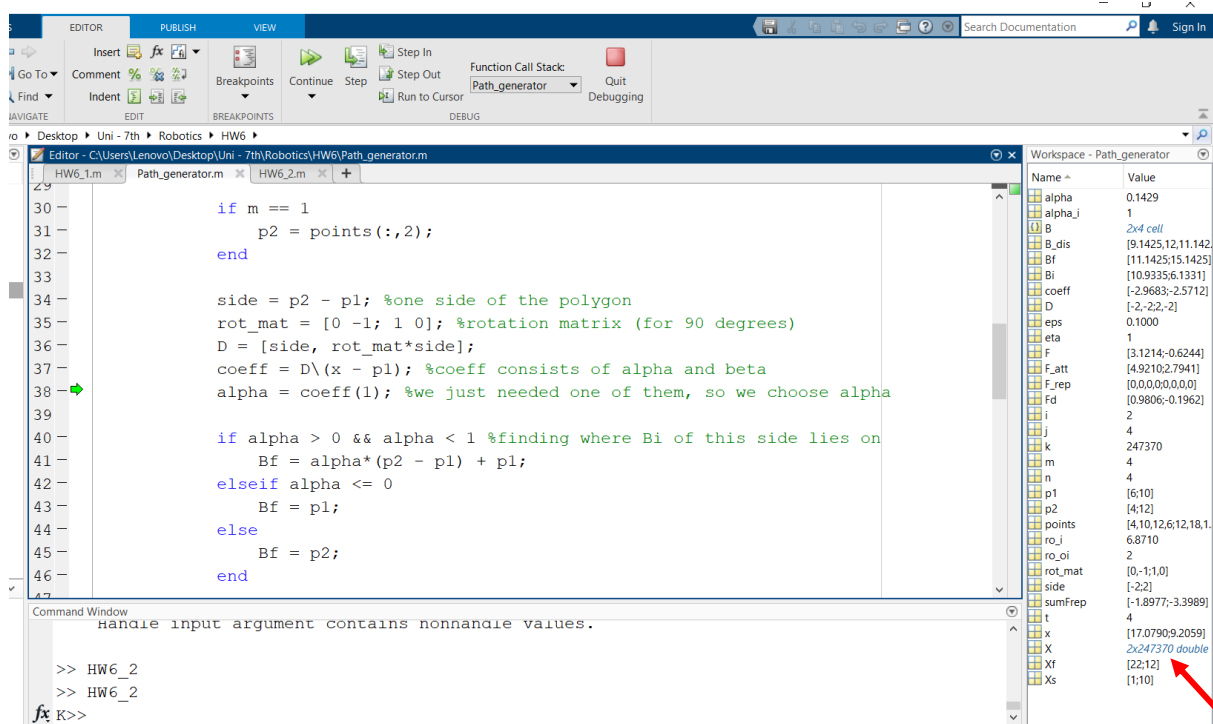
```
P = Path_generator(Xs,Xf,eta,B);
```

```
obs2_1 = polyshape(B2{2,1}(1,:), B2{2,1}(2,:));
obs2_2 = polyshape(B2{2,2}(1,:), B2{2,2}(2,:));
obs2_3 = polyshape(B2{2,3}(1,:), B2{2,3}(2,:));
obs2_4_x = B2{2,4}(1,:); obs2_4_y = B2{2,4}(2,:);
```

```
plot(obs2_1);
hold on
plot(obs2_2);
plot(obs2_3);
plot(obs2_4_x, obs2_4_y);
```

```
path_x = P(1,:); path_y = P(2,:);
plot(path_x, path_y)
```





مسیری که تا این جا تهیه شده را با دادن دستور در قسمت دیباگ رسم می‌کنیم:

```

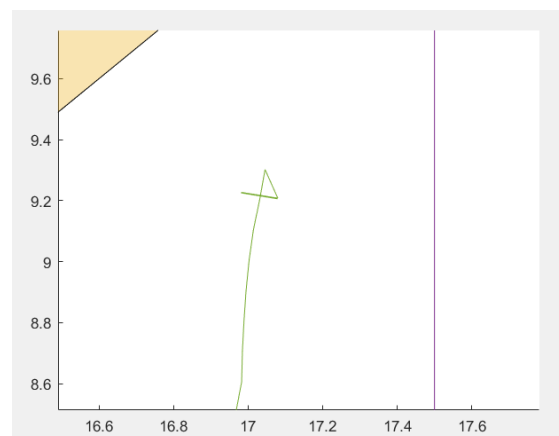
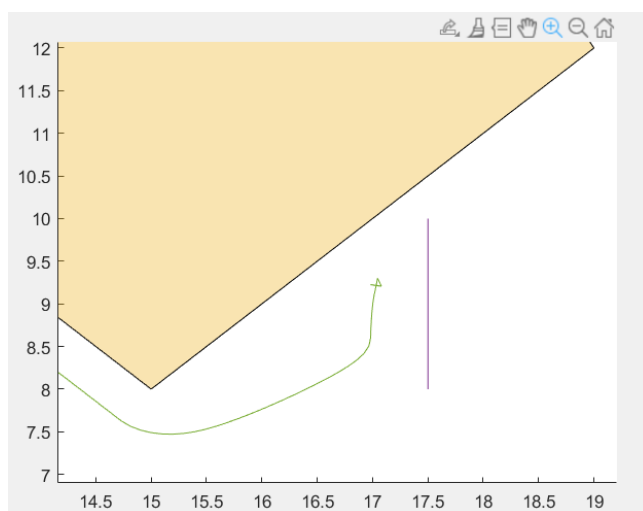
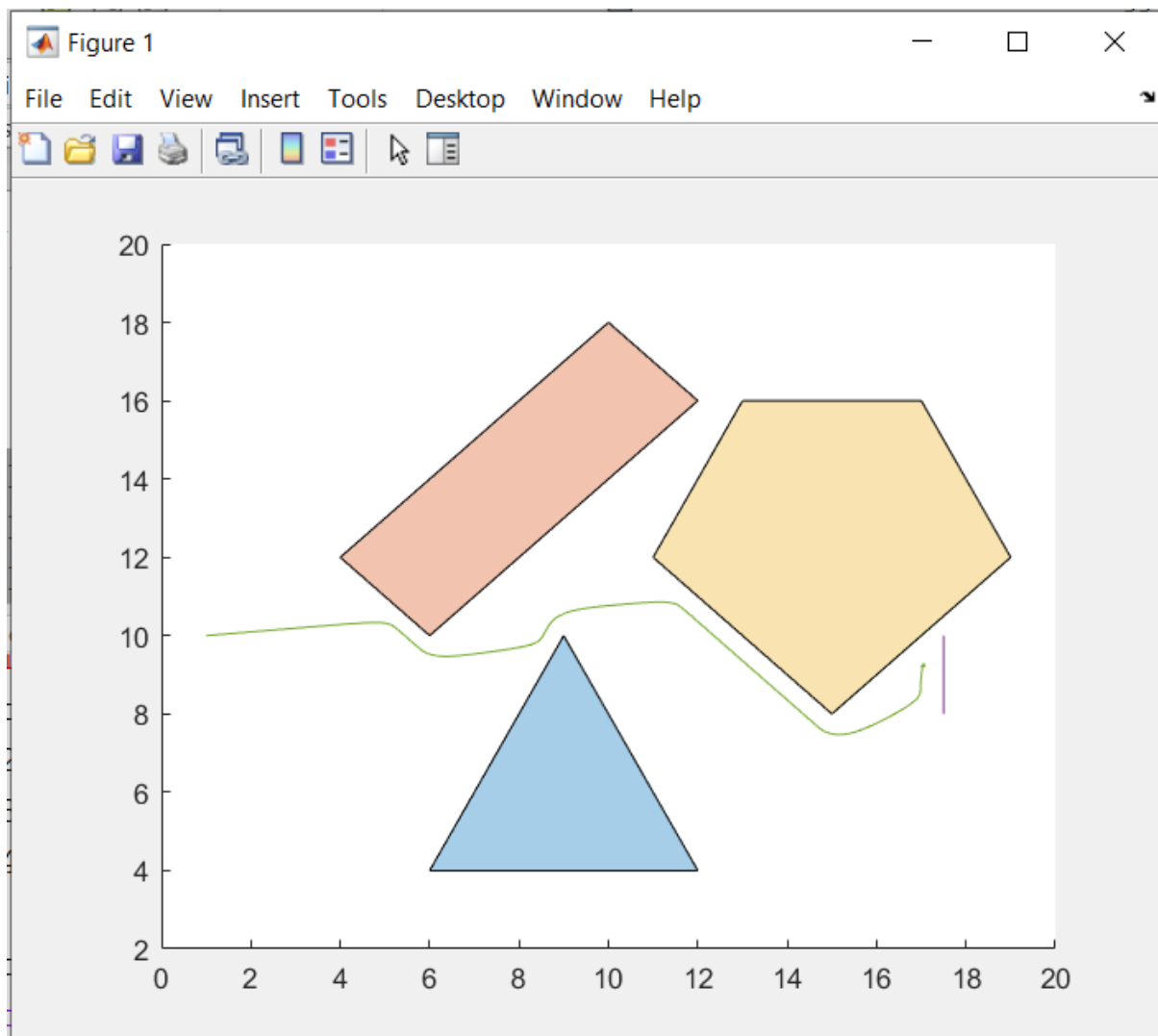
K>> obs2_1 = polyshape(B{2,1}(1,:), B{2,1}(2,:));
obs2_2 = polyshape(B{2,2}(1,:), B{2,2}(2,:));
obs2_3 = polyshape(B{2,3}(1,:), B{2,3}(2,:));
obs2_4_x = B{2,4}(1,:); obs2_4_y = B{2,4}(2,:);

plot(obs2_1);
hold on
plot(obs2_2);
plot(obs2_3);
plot(obs2_4_x, obs2_4_y);

path_x = X(1,:); path_y = X(2,:);
plot(path_x, path_y)

```

به پلات زیر خواهیم رسید:



به نظر می‌رسد که مسیر در یک مینیمم محلی گیر کرده باشد. البته با توجه به این که در مسیر شکستگی رخ داده می‌توان گفت که مسیر بین دو مانع گیر کرده است و راه رفت و برگشت ندارد و در یک لوپ رفت و برگشتی بینهایت گیر افتاده است.

۴.۲ سوال چهارم

*کدها و مدل‌ها به پیوست قرار گرفته‌اند.